Software Design Patterns

Contents

Task 1
Task 2
Task 3

Meeting objective

The objective of this meeting is to consolidate knowledge about selected design patterns and to show their use in real code – in the standard Java library and the implementation of one selected pattern.
In particular, after performing these exercises, we would like you to remember that:

1. **Design patterns are present in real production code as well as in a simple API**
2. **The class names in use do not have to and usually do not correspond to the class names from the pattern image** – what is important is the structure/dynamics, which is the essence of the pattern (although, of course, using the naming from the design pattern can facilitate communication).

Task 1

Let's try to check how the implementation of the design pattern may look like in real code. In this case, we will look into the Java standard library. The implementation of the first pattern we are going to look for is almost like in a textbook.
Let's start with downloading *Patterns* project from Moodle course, unpacking it and opening it in the *IntelliJ* environment. The *pom.xml* file is configured for Java 11 version, if you are using a different version modify properties *maven.compiler.source* and *maven.compiler.target*.

Your search should start with the *put.io.patterns.searchfor.streams* package. There is a *StreamsRunner* class containing a piece of runnable code. In simple terms, the *main* method runs code from the *run* method. However, to run a program you need to call it with a parameter that is a path to a file. To view and edit the runtime configuration right-click *StreamsRunner* and select *More Run/Debug->Modify Run Configuration*. In the *Program arguments* field, enter the path to a file. Information about the file will be shown on the screen when the program is started.

If you take a closer look at the *run* method, you'll see references to three classes: *InputStream*, *FileInputStream*, *CheckedInputStream*. Your task is to:

- **review the hierarchy of these classes** - in *IntelliJ IDE* you can view the class hierarchy in many ways, two quick methods: 1) Ctrl + click on the class name in the editor or call to its method will open a new editor tab with its source; 2) using the hierarchy view - place the cursor on the class name in the editor and select from the menu *Navigate -> Type Hierarchy* from the menu - using this view you can easily reach the superclasses and subclasses of the class you are interested in.
- **complete the received class diagram** – put your observations on the provided class diagram to be completed (available on Moodle). You can annotate them directly in *PDF* file if you have an appropriate tool or describe in a text file what should be written in the fields marked with consecutive numbers.
- **determine which design pattern they constitute** – remember that each pattern has a specific goal and start with the search for that goal. If you focus only on the structure, you have a good chance of making a mistake. So, first think what the author of the code wanted to achieve by creating a given structure in the object-oriented code?
- **determine which class plays which role defined in the design pattern** – use the names from the pattern definition (e.g. from lecture slides) for each class in its implementation you are analysing. If you are unable to do so, it is most likely that you misguessed the pattern.

Task 2

For a moment, let's give up Sherlock Holmes' approach and let's try to implement a pattern. So let's take a look at the *put.io.patterns.implement* package.

The most important class in this package for us at the moment is the *SystemMonitor* class. It can examine the state of some system resources (RAM, CPU and USB devices).

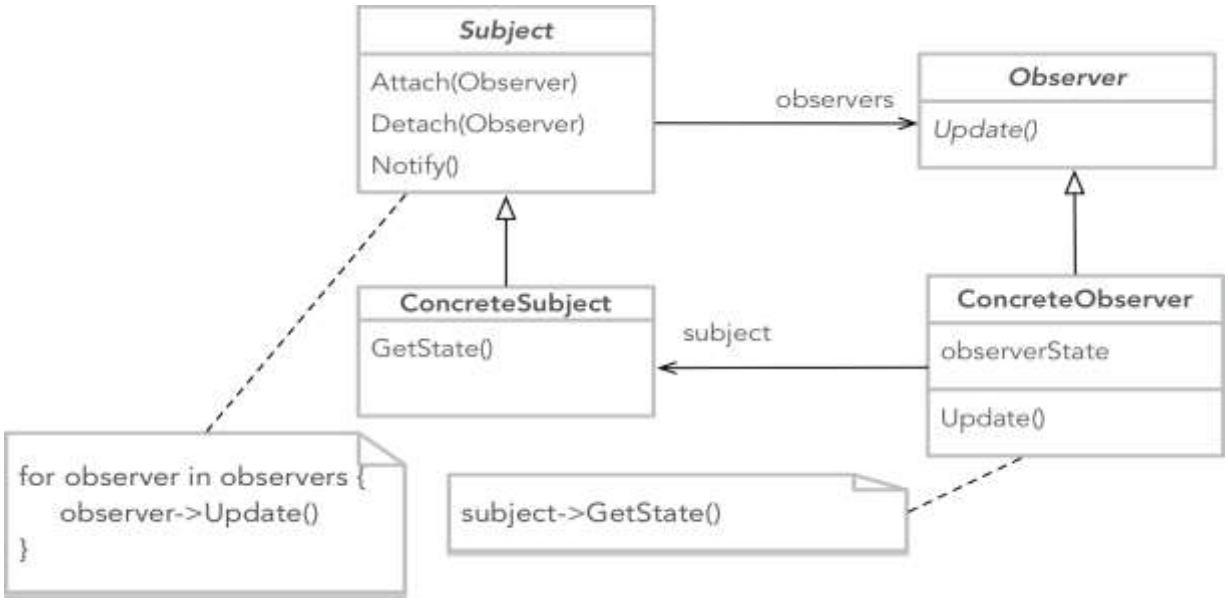
You will also find the *MonitorRunner* class implemented in this package. This class contains the *main* method, which will run our system monitor (at the moment in an infinite loop it will examine the system state with an interval of 5000 ms).

At the moment, three things are happening in the *SystemMonitor probe* method:

1. The current system status is read.
2. Information about the system status is displayed.
3. Certain rules are activated depending on the fulfilment of certain conditions (of course, we only simulate the actual actions by writing information that they took place on the screen).

I don't know if you've noticed it yet, but the *SystemMonitor* class seems to be overloaded with responsibilities: it monitors, reports, prevents...
It is definitely not well-written object code. Just think how to test these particular behaviours? Let's try to remedy this. What pattern do you think would be useful here?

One such pattern that would help us to solve our problems would be the *Observer* (sometimes called the *Listener*). Below you will find a general class diagram for this pattern (a bit more in the lecture). When analysing the class diagram, think about how you can decompose the *SystemMonitor* class to leave it with only one responsibility = **reading system information and sharing that information with others** (that's what its name comes from).



Observer design pattern.

You can probably already see that *SystemMonitor* could act as the observed object, i.e. the *Subject*. It would allow *observers* to subscribe to notifications about a change in the system state. *Observers* would be notified after each read of the system status and it would be up to each of them to respond appropriately to the information received.

If you feel up to the challenge, you can try to implement the pattern without the following tips.

Let's start by defining a common interface for all observers. Let's call it *SystemStateObserver*. The observer will have one *update* method. Each observer implementing this interface will perform operations in response to a change in the system state in this method:

```
public interface SystemStateObserver{
  public void update(SystemMonitor monitor);
}
```

While implementing the *update* method, we can pass an instance of the *SystemMonitor* class as a parameter, and then a specific observer would have to get this state from it:

```
public void update(SystemMonitor monitor){
  monitor.getLastSystemState()
...
}
```

We could also deviate a bit from the textbook pattern and pass the "handle" to the current state directly to this method:

```
public void update(SystemState newState);
```

Now, we have a good starting point. Now let's prepare our *Subject* class (*SystemMonitor*) for the possibility of admitting *observers*. Let's modify the *SystemMonitor* class (implement both methods of course):

```
public class SystemMonitor{
  private List<SystemStateObserver> observers = new ArrayList<SystemStateObserver>();
  ...
  public void addSystemStateObserver(SystemStateObserver observer){
  ...
  }
  public void removeSystemStateObserver(SystemStateObserver observer){
  ...
  }
...
}
```

Time to implement your first observer. It will be of course a separate class implementing the *SystemStateObserver* interface:

- *SystemInfoObserver* – will print a report on the resource status to the console (move the implementation directly from *SystemMonitor* → *probe*)

We also need to add a method to notify all observers that the state has changed. It will be a method in our "*Subject*" class, i.e. *SystemMonitor*:

```
public void notifyObservers(){
  for(SystemStateObserver observer : this.observers){
    observer.update(this);
  }
}
```

And of course, in the *probe* method, immediately after reading the system parameters, this method should be called:

```
public void probe(){
...
  lastSystemState = new SystemState(cpuLoad, cpuTemp, memory, usbDevices);
  notifyObservers();
}
```

So, we already have one of the blocks of the puzzle. Let's try to run it as a whole. Let's go to the *MonitorRunner* class and change the *main* method by adding the code that creates an instance of the new *observer* and adding it for notifications via the monitor:

```
public static void main(String args[]){

  // first create the monitor
  SystemMonitor monitor = new SystemMonitor();

  // create the observer and add it to the monitor
  SystemStateObserver infObserver = new SystemInfoObserver();
  monitor.addSystemStateObserver(infObserver);

  while (true) {
    monitor.probe();
    try {
      Thread.sleep(5000);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
  }
}
```

It's time to implement additional *observers* – each one will be a separate class implementing the *SystemStateObserver* interface, using the existing code transferred from the *probe* method in *SystemMonitor* (if the appropriate code is there):

- *SystemGarbageCollectorObserver* – will run the garbage collector when memory is low.
- *SystemCoolerObserver* - will start cooling if the processor overheats.
- *USBDeviceObserver* - this is something new, let's try to make an observer that will print a message only if the number of USB devices has increased or decreased (the observer must remember recently received watched states from *SystemMonitors*).

Well done! The first pattern has been successfully implemented. Thanks to the use of the *observer* pattern, we have reduced the *SystemMonitor* class responsibility. We have also opened the way to add new functionality, which will be related to the response to changes in the state of system resources. Also note that it would be much easier for us now to test if the behaviour in response to the system state change is correct because we can test each observer separately from *SystemMonitor*.

Task 3

Let's go back to looking for patterns in Java code. This time, let's go to the *put.io.patterns.searchfor.sierpinski* package. The program is in the *SierpinskiRunner* class.

The *SierpinskiRunner* configuration has a default depth of 3. To change the depth, right click *SierpinskiRunner* and *Modify Run Configuration*, select a value within reasonable limits (2-4).

Of course, displaying the *Sierpinski* carpet is not important to us at this point. Just like task 1, make an exploration of the class hierarchy. Also review the implementations of the indicated methods. This time we are looking for two patterns. The implementation of pattern 2 is quite textbook-like, while pattern 1 has already visible differences, which in this case result to some extent from the historical conditions of Java development (the pattern penetrates through two "libraries" - the older and "lower" level *AWT* and the later "higher" level called *Swing*). Remember to define the role depending on the pattern (in the provided PDF with diagrams, the place of the two patterns is explicitly separated).